ДЗ 2 · EFFICIENT DEEP LEARNING

Exclusive Self Attention

Разбор статьи и воспроизведение на маленьком GPT

S. Zhai (Apple), arXiv:2603.09078, март 2026

Как устроен self-attention

$$y_i = \sum_{j \le i} a_{ij}\, v_j, \qquad a_{ij} = \mathrm{softmax}_j\left(\frac{q_i^\top k_j}{\sqrt{d}}\right)$$

$$x \leftarrow x + \mathrm{Attn}(\mathrm{LN}(x)), \qquad x \leftarrow x + \mathrm{FFN}(\mathrm{LN}(x))$$

Сам токен и так проходит в следующий слой по residual-связи, а FFN обрабатывает каждую позицию отдельно. От attention нужна информация из других позиций.

СТАТЬЯ

Attention similarity bias

Автор взял свою обученную модель на 1.3B и посчитал по слоям:

- косинус между выходом внимания y_i и своим value v_i: от 0.2 до 0.55, к последним слоям растёт;
- косинус между value разных токенов: положительный, 0.04–0.10;
- вес внимания на себя a_{ii}: до 0.13 при длине 2048.

Выход внимания во многом повторяет то, что у токена уже есть. Автор считает, что attention тратит на это ёмкость и конкурирует с FFN вместо того, чтобы собирать контекст.

XSA: вычесть из выхода свой value

$$z_i = y_i - \frac{y_i^\top v_i}{\|v_i\|_2^2} v_i$$

z_i — проекция y_i на подпространство, ортогональное v_i. Остаётся то, что токен получил из контекста и чего нет в его собственном value.

```
Y  = F.scaled_dot_product_attention(Q, K, V, is_causal=True)
Vn = F.normalize(V, dim=-1)                              # XSA
Z  = Y - (Y * Vn).sum(dim=-1, keepdim=True) * Vn         # XSA
```

Что из этого следует

Параметров не прибавилось

Меняются две строки после softmax-внимания.

$z_i \perp v_i$

Похожесть выхода на свой value убрана полностью, по построению.

Первый токен ничего не получает

Он видит только себя: $y_1 = v_1$, значит $z_1 = 0$.

Неявный attention sink

Если голова смотрит на себя, XSA всё вычтет. Так голова может ничего не выдавать.

Цена

Несколько поэлементных операций над тензором B×T×d. Матричные умножения в блоке стоят B×T×d², так что на больших моделях добавка теряется.

Эксперименты автора

Модели	0.7B, 1.4B, 2.7B без эмбеддингов
Данные	FineWeb, 100B токенов
Обучение	nanoGPT с RoPE, контекст 2048, 200K шагов по 0.5M токенов
Скорость	блок attention+FFN на B200

Среднее по 8 задачам	baseline	XSA
0.7B	53.22	53.48
1.3B	56.16	57.19
2.7B	58.06	59.42

Табл. 2 статьи

Loss ниже на 0.02–0.03 при всех размерах, разница одинакова при разных LR, растёт с длиной контекста от 512 до 16384 и сохраняется с attention sinks. Время и память почти не растут.

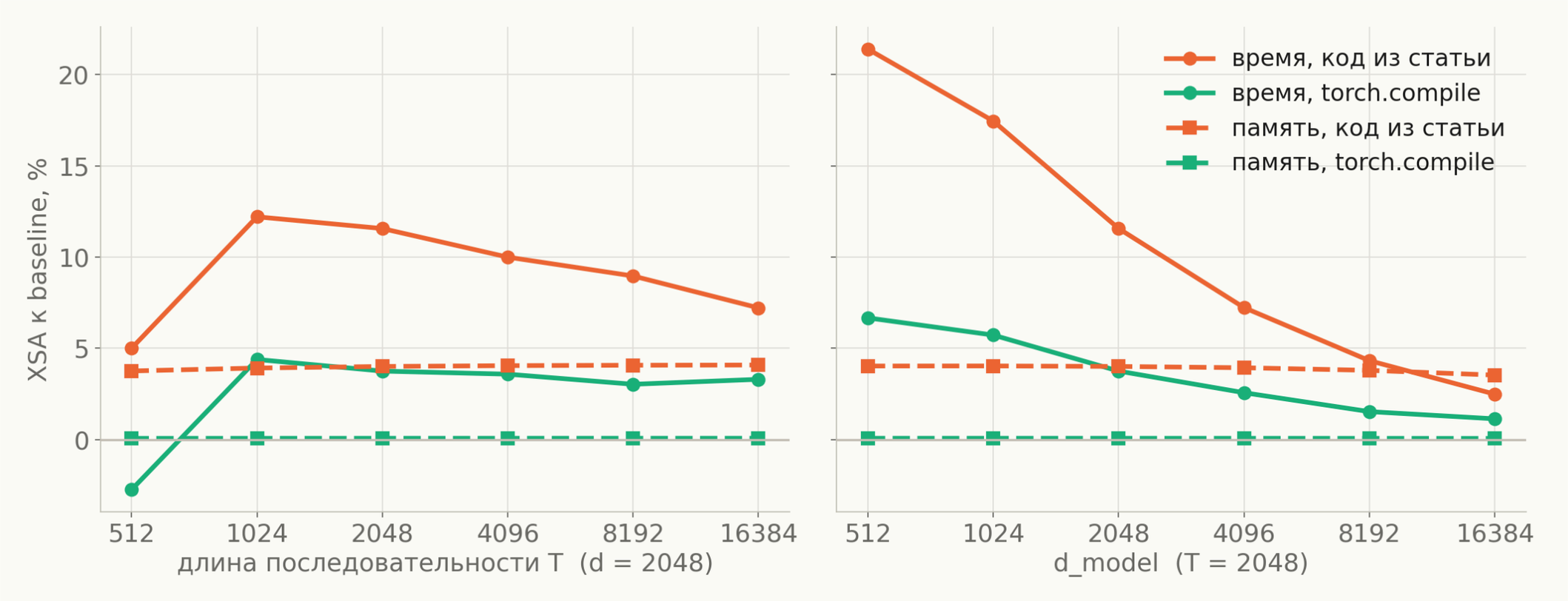
Что я проверял

	Статья	Здесь
Модель	0.7–2.7B без эмбеддингов	основная 11M без эмбеддингов (30M всего), 6 слоёв, d=384; ещё 3M и 25M
Токенов на запуск	100B	100M, FineWeb
Контекст	2048	1024, ещё 256 и 4096
Железо	B200	A100 80GB в Colab, bf16, torch.compile
Повторы	1 запуск	3 сида на основную модель и длины контекста, всего 33 запуска

При одном сиде baseline и XSA стартуют с одних и тех же весов и видят одни и те же батчи. Разница в loss внутри пары зависит только от XSA.

ВОСПРОИЗВЕДЕНИЕ · РИС. 2

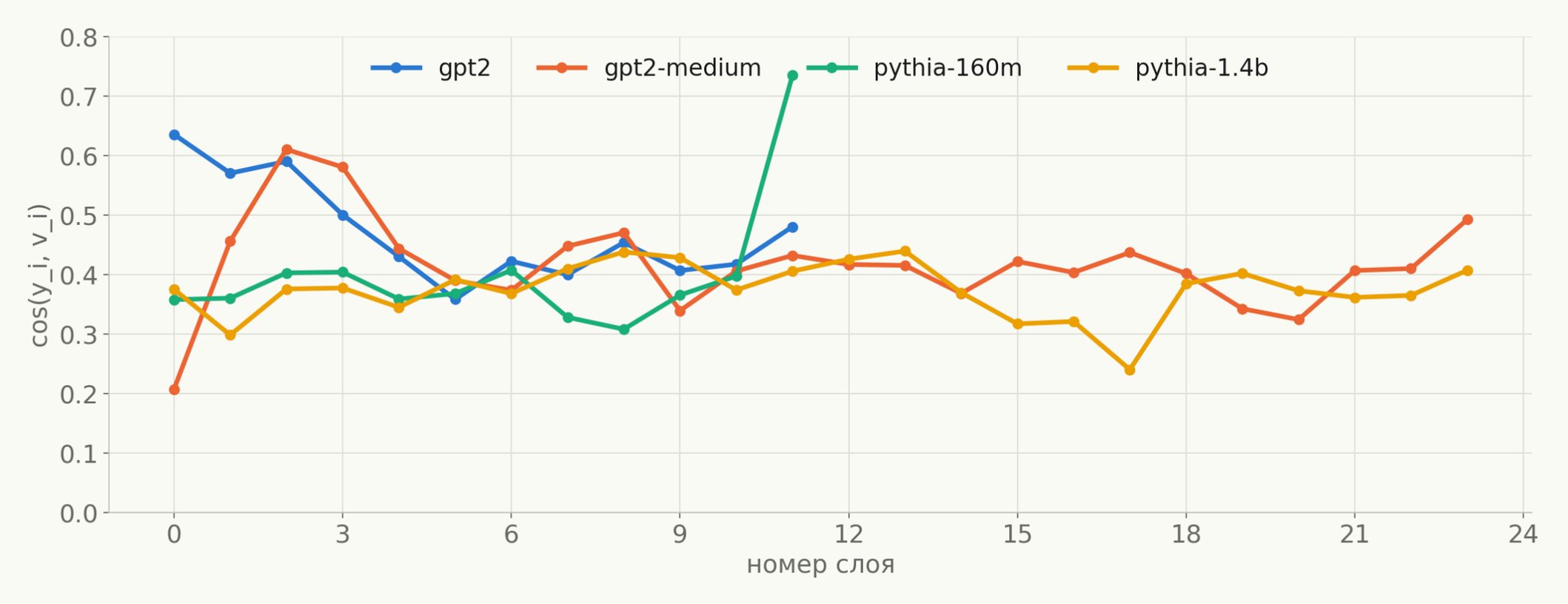
Время и память



Код из статьи: +5…12% времени блока при d=2048 и +4% памяти. Через torch.compile +1…4%, память не растёт. Сама операция упирается в память: при T=16384 eager идёт 89 мс, а одно fused-ядро 22 мс. При обучении шаг дольше на 2–4%.

ВОСПРОИЗВЕДЕНИЕ · РИС. 1

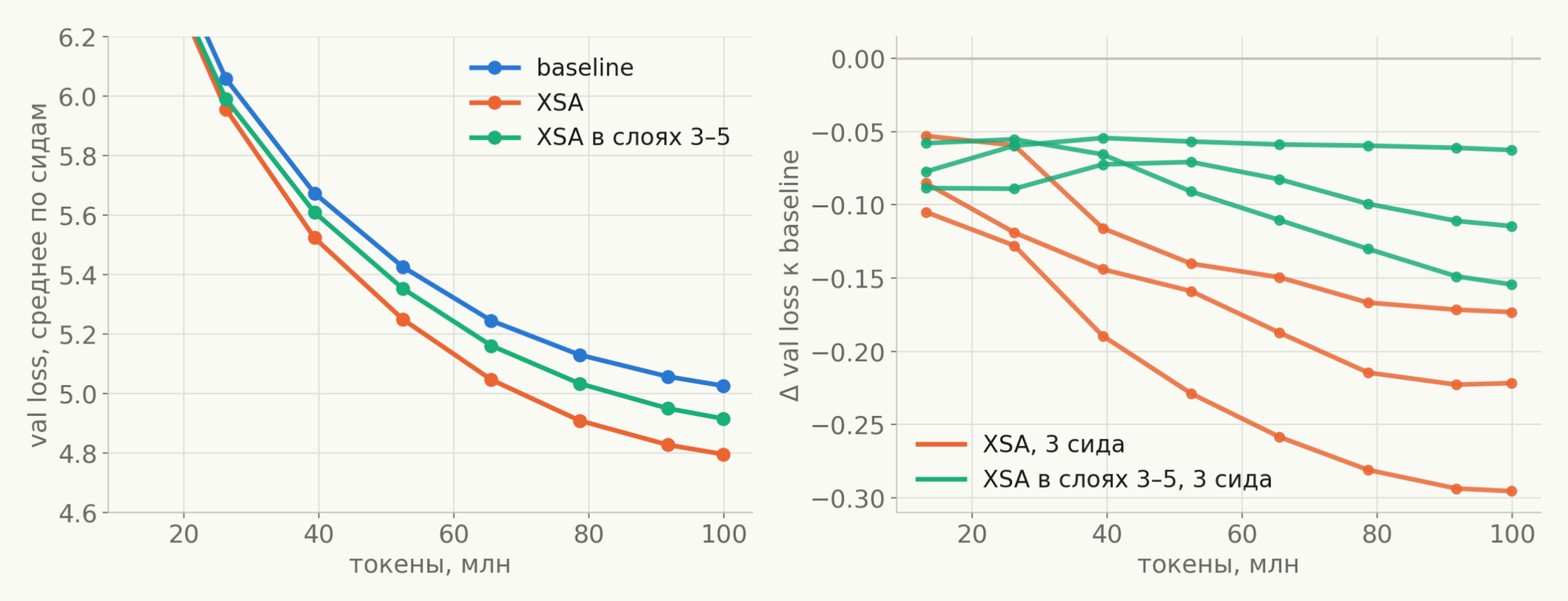
Similarity bias у открытых моделей



Модель автора не выложена, поэтому смотрел GPT-2 и Pythia. У всех $\cos(y_i, v_i)$ около 0.4, у случайных векторов было бы около нуля. Роста к последним слоям, как у автора, у Pythia-1.4B нет.

ВОСПРОИЗВЕДЕНИЕ · РИС. 3

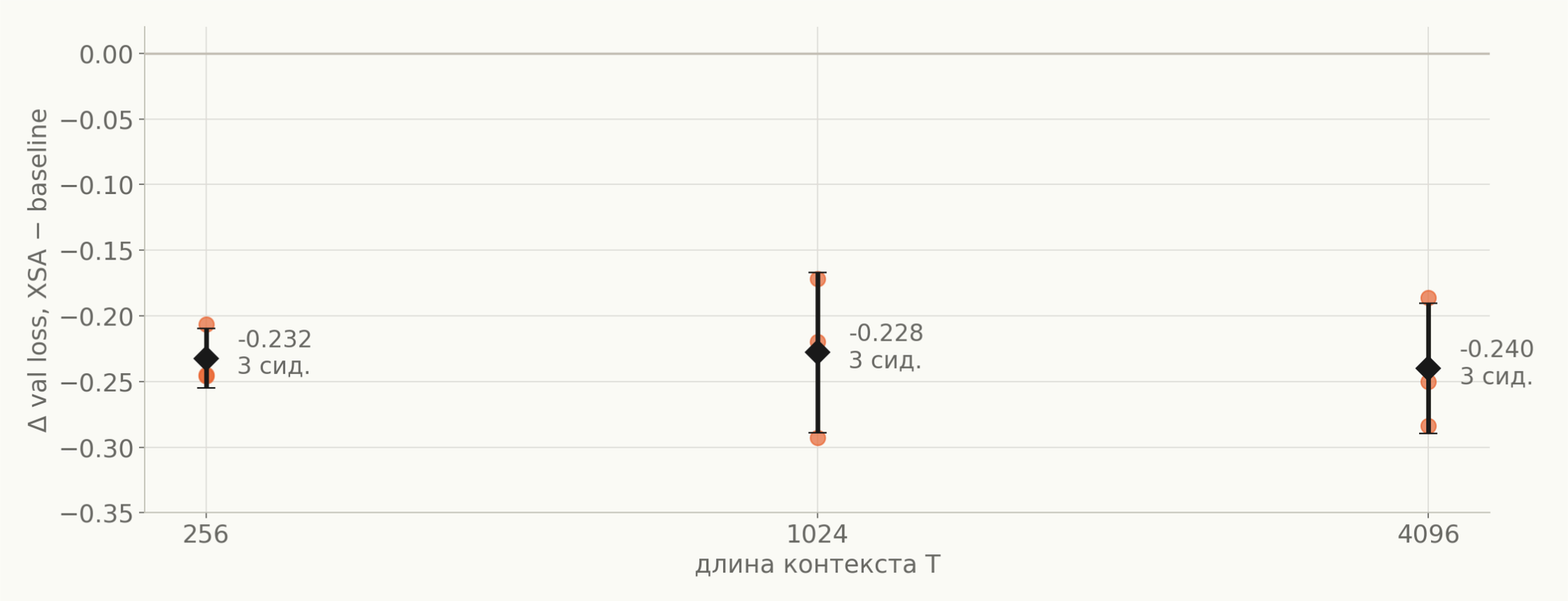
Loss: XSA лучше на всех сидах



Итоговый val loss: 5.004 у baseline и 4.776 у XSA, **Δ = −0.23 ± 0.06**. XSA только в верхних трёх слоях даёт половину выигрыша, −0.11 ± 0.05. Во всех 18 парах прогона XSA лучше baseline. У автора разница около 0.02.

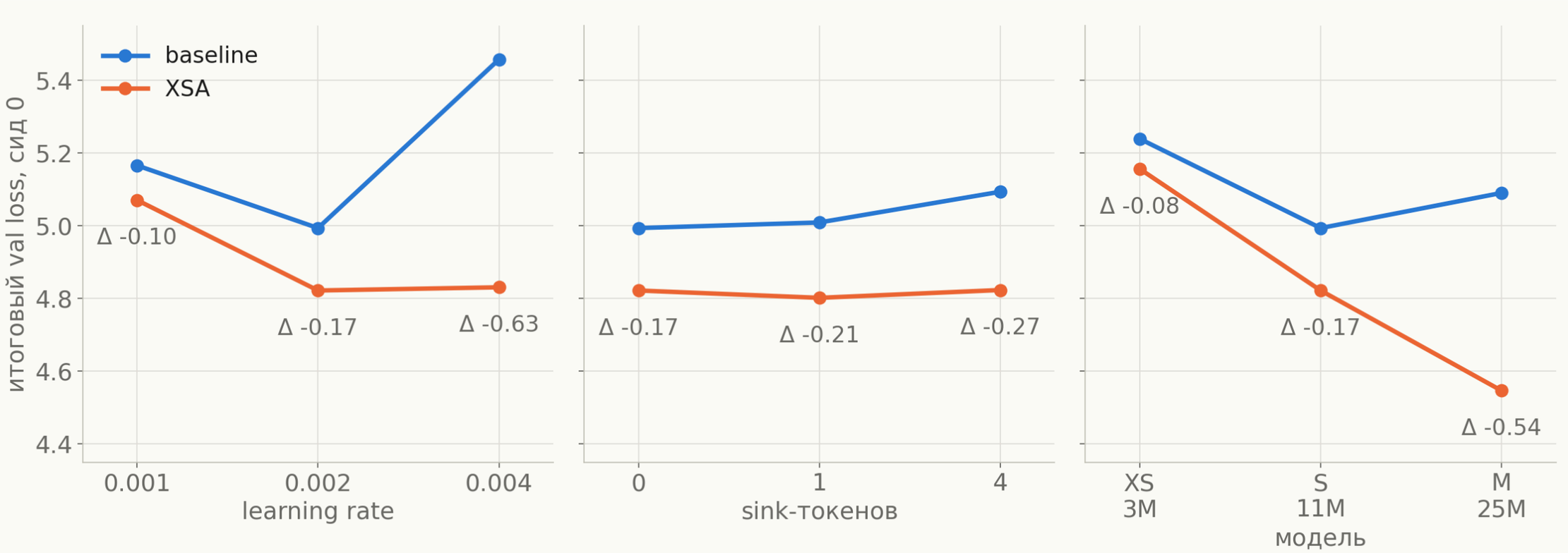
ВОСПРОИЗВЕДЕНИЕ · РИС. 5

Длина контекста: роста нет



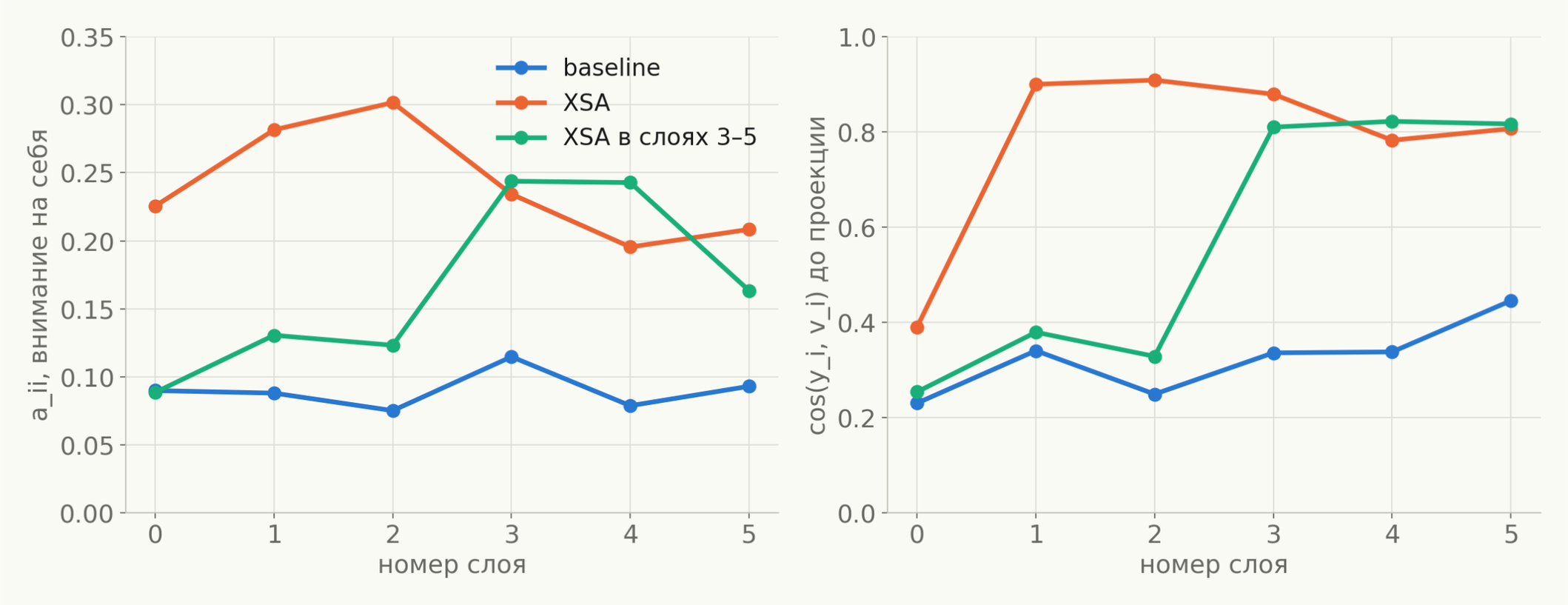
По 3 сида на каждую длину, выигрыш одинаковый. Утверждение статьи здесь не воспроизвелось. Вероятно, 100M токенов мало: при T=4096 loss у обеих моделей даже выше, чем при 1024.

LR, sinks и размер модели



XSA лучше везде. Но baseline сильно проседает при большом LR, а XSA нет: отсюда −0.63 при LR 4e-3 и −0.54 у модели M, для которой 2e-3 уже великовато. Обучаемые sink-токены baseline не помогают.

Что выучивает модель с XSA



Слои с XSA смотрят на себя в 2–4 раза чаще, а y_i почти сонаправлен с v_i. Эту часть XSA вычитает, так голова «молчит»: это и есть неявный sink из статьи. Если XSA стоит только в верхних слоях, меняются только они.

ИТОГ

Что воспроизвелось

Утверждение статьи	У нас	Итог
Время и память почти не растут	+5–12% времени блока, через torch.compile +1–4%	да
Выход внимания похож на свой value	$\cos(y_i, v_i)$ около 0.4 у GPT-2 и Pythia	да, но без роста по слоям
XSA снижает loss	−0.23 ± 0.06, лучше во всех 18 парах	да, сильнее, чем у автора
Выигрыш растёт с размером	−0.08, −0.17, −0.54 для 3M, 11M, 25M	да, но у 25M baseline мешает LR
Разница не зависит от LR	−0.10, −0.17, −0.63 при LR 1e-3, 2e-3, 4e-3	нет, baseline чувствительнее к LR
Растёт с длиной контекста	−0.23, −0.23, −0.24 при T = 256, 1024, 4096	нет
Работает с attention sinks	−0.17, −0.21, −0.27 при 0, 1, 4 sink-токенах	да
Лучше на downstream	LAMBADA 0.1% → 0.9%, HellaSwag на уровне случайного	на таком масштабе не проверить